

P3971R1

std::rebind - Generalized Type Rebinding for Containers and Uniform-Element Types

Published Proposal, 2026-02-20

This version:

<http://wg21.link/P3971>

Author:

[Daniel Towner](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes `std::rebind`, a facility for converting containers and other types to use a different element type while preserving structure. This enables generic programming patterns that work uniformly across `array`, `vector`, `complex`, and user-defined types, filling a gap left by the lack of a generalized rebinding mechanism.

Table of Contents

- 1 Revision History**
- 2 Motivation**
 - 2.1 Precedent in `std::simd`
 - 2.2 Proposed Solution
- 3 Supported Types**

- 3.1 Specification Principle
- 3.2 Standard Library Support
- 3.3 User-Defined Types
- 3.4 Excluded Types

4 Examples

- 4.1 Basic Usage
- 4.2 Generic Type Conversion
- 4.3 Complex Numbers
- 4.4 User-Defined Types

5 Design Alternatives

- 5.1 Range Adaptor Syntax
- 5.2 Tuple and Pair
- 5.3 Container Adaptors
- 5.4 Associative Containers
- 5.5 Naming Alternatives
- 5.6 Customisation Mechanism
- 5.7 Relationship Between `rebind_t` and `rebind`
- 5.8 Value-Preserving Conversions
- 5.9 Duration and Units Types

6 Wording

- 6.1 [utility.rebind] Generalised element-type conversion
 - 6.1.1 Type trait `rebind_t`
 - 6.1.2 Customisation point object `rebind`

References

Informative References

§ 1. Revision History

R0

- Initial revision

§ 2. Motivation

Modern C++ provides powerful facilities for generic programming, but lacks a uniform way to change the element type of containers and container-like types. Consider a simple requirement: convert a container of `float` values to `double` for higher precision computation.

For `std::vector`, this is straightforward using constructor-based conversion:

```
std::vector<double> widen(const std::vector<float>& data) {  
    return std::vector<double>(data.begin(), data.end());  
}
```

However, making this work for `std::array` requires completely different code:

```
template<std::size_t N>  
std::array<double, N> widen(const std::array<float, N>& data) {  
    std::array<double, N> result;  
    std::copy(data.begin(), data.end(), result.begin());  
    return result;  
}
```

For `std::complex`, yet another approach is needed:

```
std::complex<double> widen(const std::complex<float>& data) {  
    return std::complex<double>(data);  
}
```

Each type requires specialised knowledge of its conversion mechanisms. Attempting to write a single generic function fails:

```
template<typename Container>  
auto widen_to_double(const Container& data) {  
    using T = typename Container::value_type; // Some container types may  
  
    // How do we create Container<double> from Container<float>?  
    // - vector: range constructor  
    // - array: manual copy with known size  
    // - complex: direct construction  
    // - user types: ???
```

```
    // No uniform solution exists
}
```

§ 2.1. Precedent in `std::simd`

The `std::simd` proposal [P1928R15] recognised this problem and introduced `rebind_t` as a type trait to convert the type `basic_simd<T, Abi>` to `basic_simd<U, Abi>`. This works well for `simd`, but the mechanism is not generalised to other types. The `simd` proposal provides both a type trait for compile-time type computation and suitable conversion constructors which make using that type to change the underlying type easy. No other current container can do the same.

Recent discussion of `simd` casting utilities [P3445R0] raised questions about whether such facilities should be generalised beyond `simd` to support other types like containers and units. This proposal explores that generalisation.

§ 2.2. Proposed Solution

We propose two complementary facilities:

1. `rebind_t<U, T>` - A type trait that computes the result type of rebinding the structure from elements of type `T` to use elements of type `U`
2. `std::rebind<U>(obj)` - A customisation point object that performs the actual conversion

These work together to enable uniform generic code:

```
template<typename Container>
auto widen_to_double(const Container& data) {
    using T = typename Container::value_type;
    return std::rebind<double>(data);
}

// Works uniformly for all supported types:
std::vector<float> v = {1.0f, 2.0f, 3.0f};
auto vd = widen_to_double(v); // vector<double>

std::array<float, 3> a = {1.0f, 2.0f, 3.0f};
auto ad = widen_to_double(a); // array<double, 3>
```

The facilities are extensible via ADL, allowing user-defined types to participate in generic algorithms using the same interface.

§ 3. Supported Types

§ 3.1. Specification Principle

`std::rebind` is provided for types where rebinding the element type produces a meaningful corresponding type. The presence of a `value_type` member typedef is a strong indicator that a type has a uniform element type suitable for rebinding, but is neither necessary nor sufficient:

- Types like `std::complex<T>` are clearly rebindable despite lacking `value_type`.
- Types like `std::stack<T>` have `value_type` but cannot be rebound without additional complexity (rebinding the underlying container).
- Associative containers have `value_type` but require also rebinding their comparator.

The determination of whether a type is rebindable depends on whether the rebinding operation is well-defined and produces a semantically equivalent structure with a different element type.

Standard library support is provided for:

1. **Sequence containers:** Types with `value_type` representing a sequence of uniform elements
2. **Scalar types with uniform components:** Types like `std::complex` that represent a single value composed of uniform components
3. **User-defined types:** Via ADL customisation for types where rebinding is appropriate

§ 3.2. Standard Library Support

The following table shows the standard library types for which `std::rebind` and `rebind_t` would be defined:

Input type	Has <code>value_type</code> ?	Result of rebinding to <code>double</code>
<code>std::array<T, N></code>	Yes	<code>std::array<double, N></code>
<code>std::vector<T, A></code>	Yes	<code>std::vector<double, rebind_alloc_t<A, double>></code>

Input type	Has <code>value_type</code> ?	Result of rebinding to double
<code>std::deque<T, A></code>	Yes	<code>std::deque<double, rebind_alloc_t<A, double>></code>
<code>std::list<T, A></code>	Yes	<code>std::list<double, rebind_alloc_t<A, double>></code>
<code>std::forward_list<T, A></code>	Yes	<code>std::forward_list<double, rebind_alloc_t<A, double>></code>
<code>std::complex<T></code>	No (special case)	<code>std::complex<double></code>

For allocator-aware containers, the allocator is automatically rebound using `rebind_alloc_t<Allocator, U>`, following existing standard library practice.

§ 3.3. User-Defined Types

User-defined types can provide `rebind` support via ADL by defining a `rebind` function in the same namespace as the type:

```
namespace mylib {
    template<typename T>
    struct Vec3 {
        using value_type = T; // Strong hint that rebind makes sense.
        T x, y, z;
    };

    template<typename U, typename T>
    Vec3<U> rebind(const Vec3<T>& v) {
        return Vec3<U>{static_cast<U>(v.x), static_cast<U>(v.y), static_ca
    }
}

// rebind_t would also work.
static_assert(std::is_same_v<std::rebind_t<double, mylib::Vec3<float>>,
mylib::Vec3<double>>);
```

§ 3.4. Excluded Types

Associative containers (`set`, `map`, etc.) are not supported because rebinding the element type requires also rebinding the comparator type, which lacks a general solution.

Tuple-like types (`tuple`, `pair`) are excluded because they are fundamentally heterogeneous types. See [§ 5.2 Tuple and Pair](#) for details.

Container adaptors (`stack`, `queue`, `priority_queue`) are not included in initial support. See [§ 5.3 Container Adaptors](#) for details.

Duration and units types (e.g., `std::chrono::duration`) are not included in the initial standard library support due to ambiguity about what should be rebound (see [§ 5.9 Duration and Units Types](#)), but such types can provide `rebind` support via the ADL customisation mechanism if appropriate.

§ 4. Examples

§ 4.1. Basic Usage

Converting element types is straightforward with `std::rebind`:

```
// Arrays – preserves size
std::array<float, 4> af = {1.0f, 2.0f, 3.0f, 4.0f};
std::array<double, 4> ad = std::rebind<double>(af);

// Type computation
using ArrayDouble4 = std::rebind_t<double, decltype(af)>;
static_assert(std::is_same_v<ArrayDouble4, std::array<double, 4>>);

// Vectors – preserves allocator
std::vector<int> vi = {1, 2, 3, 4, 5};
std::vector<long> vl = std::rebind<long>(vi);

// Complex numbers
std::complex<float> cf{3.0f, 4.0f};
std::complex<double> cd = std::rebind<double>(cf);
```

§ 4.2. Generic Type Conversion

Structure-preserving type conversion works uniformly across container types:

```
template<typename U, typename Container>
auto convert_elements(const Container& c) {
    return std::rebind<U>(c);
}

std::array<int, 5> ints = {1, 2, 3, 4, 5};
auto doubles = convert_elements<double>(ints); // array<double, 5>

std::vector<float> floats = {1.0f, 2.0f};
auto longs = convert_elements<long>(floats); // vector<long>
```

§ 4.3. Complex Numbers

`rebind` works uniformly for `complex`:

```
// Complex number precision conversion
std::complex<float> cf{3.0f, 4.0f};
auto cd = std::rebind<double>(cf); // complex<double>{3.0, 4.0}

// Useful for mixed-precision algorithms
template<typename T>
auto high_precision_norm(const std::complex<T>& c) {
    auto hp = std::rebind<long double>(c);
    return std::abs(hp); // Computed in higher precision
}
```

§ 4.4. User-Defined Types

Users can extend `rebind` to their own types via ADL:

```
namespace mylib {
    template<typename T>
    struct Vec3 {
        using value_type = T; // Strong hint that rebinding is allowed.
    }
}
```



```

    T x, y, z;
};

// Provide rebind via ADL
template<typename U, typename T>
Vec3<U> rebind(const Vec3<T>& v) {
    return Vec3<U>{
        static_cast<U>(v.x),
        static_cast<U>(v.y),
        static_cast<U>(v.z)
    };
}
}

// Now Vec3 works with generic code:
mylib::Vec3<float> vf{1.0f, 2.0f, 3.0f};
auto vd = convert_elements<double>(vf); // Vec3<double>

// And rebind_t works automatically:
using Vec3d = std::rebind_t<double, mylib::Vec3<float>>;
static_assert(std::is_same_v<Vec3d, mylib::Vec3<double>>);

```

§ 5. Design Alternatives

§ 5.1. Range Adaptor Syntax

`std::rebind` could support range adaptor syntax to enable composition with other range adaptors:

```
auto result = input | std::rebind<double> | std::views::take(10);
```

This would follow the pattern of modern C++ range adaptors and enable natural pipelines. However, this functionality is deferred to future work to keep the initial proposal focused on core functionality. The direct function call syntax `std::rebind<T>(x)` is sufficient for the primary use cases. If LEWG finds the facility valuable, range adaptor support can be added in a subsequent revision.

§ 5.2. Tuple and Pair

`std::tuple` and `std::pair` are excluded because they are fundamentally heterogeneous types, even when all their element types happen to be the same.

Consider:

```
std::tuple<int, int, int> t = {1, 2, 3};
std::pair<int, int> p = {1, 2};

auto t2 = std::rebind<double>(t); // What does this mean?
auto p2 = std::rebind<double>(p); // Which elements to rebind?
```

The problem is not whether the types are currently the same, but that tuples (including pair, which is a 2-element tuple) are designed to hold potentially different types at each position. Without additional context specifying which elements to rebind, the operation is not well-defined. Unlike containers where all elements have the same type by design, tuples explicitly support different types at different positions. This fundamental difference makes rebinding semantically unclear.

Users who need to convert tuple or pair elements can explicitly construct the target type:

```
std::pair<int, int> p = {1, 2};
std::pair<double, double> pd{static_cast<double>(p.first),
                             static_cast<double>(p.second)};
```

§ 5.3. Container Adaptors

Container adaptors (`std::stack`, `std::queue`, `std::priority_queue`) wrap underlying containers and present implementation challenges: the underlying container type is a template parameter that must also be rebound, and `priority_queue` has a comparator (same problem as associative containers).

This proposal excludes container adaptors from initial support. They can be added in future revisions once implementation experience is gained. Users who need to rebind adapted containers can rebind the underlying container and construct a new adaptor.

§ 5.4. Associative Containers

`std::set`, `std::map`, and related associative containers are excluded from this proposal because rebinding their element type requires also rebinding their comparator type, which lacks a general solution.

Consider:

```
std::set<float, MyFloatCompare> s = /*...*/;
auto s2 = std::rebind<double>(s); // What comparator should s2 use?
```

The result type would need to be `std::set<double, ??? >`. Options include:

1. ****Use `std::less<double>`**** - Loses the custom comparator, changes semantics
2. **Attempt to rebind the comparator** - No general mechanism exists for this; comparators may not be templated
3. **Require users to provide the comparator** - Defeats the purpose of `rebind` being simple

None of these solutions is satisfactory. The comparator rebinding problem is orthogonal to element type rebinding and requires separate consideration. For now, associative containers are excluded.

A future proposal could address this by:

- Defining a `rebind_comparator` mechanism for comparators
- Extending `rebind` to accept optional comparator arguments
- Restricting support to comparators known to be rebindingable (e.g., `std::less`)

Such extensions are beyond the scope of this proposal.

§ 5.5. Naming Alternatives

Several names were considered:

- `rebind`: Chosen for consistency with existing standard library rebinding facilities
- `value_cast`: Suggests only a value conversion, but doesn't capture structural aspects like allocator rebinding
- `element_cast`: Clear but verbose; "element" may be ambiguous for non-containers like `complex`

- `convert`: Too generic; doesn't convey the type transformation aspect

We chose `rebind` because it has strong precedent in the standard library and accurately describes the operation. The term "rebind" is already well-established for analogous type transformations:

1. **Allocator rebinding** (`std::allocator_traits::rebind_alloc`, `rebind_alloc_t`):
Converting an allocator from one element type to another, e.g.,
`rebind_alloc_t<std::allocator<int>, double>` produces
`std::allocator<double>`.
2. **SIMD rebinding** (`rebind_t` from [P1928R15]): Converting a simd type from one element type to another, e. g., `rebind_t<float, basic_simd<int, Abi>>` produces
`basic_simd<float, Abi>`.

This proposal extends that established vocabulary to containers and other uniform-element types. The parallel structure reinforces conceptual consistency:

- `rebind_alloc_t<U, Allocator>` - rebind allocator's element type
- `rebind_t<U, basic_simd<T, Abi>>` - rebind simd's element type
- `rebind_t<U, Container>` - rebind container's element type

Furthermore, this proposal internally uses `rebind_alloc_t` when rebinding allocator-aware containers, creating a natural connection between the facilities.

The naming is thus not novel, but rather a systematic extension of existing practice to a broader set of types.

§ 5.6. Customisation Mechanism

The proposal uses a customisation point object with ADL lookup, following the precedent of `std::swap` and other customisable standard library operations. We also provide the `rebind_t` type trait for compile-time type computations. This approach provides the right balance of usability, extensibility, and consistency with existing practice.

§ 5.7. Relationship Between `rebind_t` and `rebind`

The type trait `rebind_t` is defined in terms of the function `rebind`:

```
template<typename U, typename T>
using rebind_t = decltype(rebind<U>(std::declval<T>()));
```

This ensures consistency: the type trait always produces the same type that the function returns. An alternative would be to define them independently, but that creates potential for inconsistency and requires duplicating customisation mechanisms.

§ 5.8. Value-Preserving Conversions

This proposal does not impose special requirements for value-preserving conversions beyond standard C++ conversion rules. The element-wise conversion from `T` to `U` uses standard conversion semantics, including narrowing conversions when requested.

```
std::vector<double> vd = {3.14, 2.71};
auto vi = std::rebind<int>(vd); // OK - explicitly requested narrowing
```

This follows the precedent of `std::simd` [P1928R15], which uses constructor explicitness to control conversion safety. In `simd`:

- Non-explicit constructors allow implicit conversions (i.e., conversion must be widening/value-preserving)
- Explicit constructors permit narrowing conversions

For `rebind`, the explicit function call is the signal that the user wants the conversion, including narrowing conversions. This is different from implicit conversions where value-preservation rules would be appropriate.

An alternative would be to require value-preserving conversions by default and provide a `flag_convert` parameter (following `simd`'s pattern for range load/store operations) to opt into narrowing:

```
std::rebind<int>(vd); // Error - narrowing
std::rebind<int>(vd, std::flag_convert); // OK - explicit opt-in
```

However, this adds complexity without clear benefit:

- `rebind` is always an explicit function call - the user already opted in
- For generic code, the caller controls what conversions are valid
- Standard C++ conversion rules already provide appropriate diagnostics (e.g., for explicit constructors)
- User-defined types can use `explicit` constructors to control conversion safety

The current design is simpler and follows the principle that explicit calls indicate explicit intent.

§ 5.9. Duration and Units Types

Types representing quantities with units, such as `std::chrono::duration` or quantity types from units proposals like [\[P3045R1\]](#), are not included in the initial standard library support for this proposal.

Such types present ambiguity about what should be rebound:

- For `std::chrono::duration<Rep, Period>`, rebinding could mean changing the representation type (Rep) or the units (Period)
- The semantics of rebinding are unclear without additional context

Additionally, `std::chrono::duration_cast` already provides conversion functionality for duration types.

However, the ADL customisation mechanism allows such types to provide their own `rebind` support if appropriate semantics can be established. Future proposals (including units proposals) can define `rebind` functions for their types without requiring changes to this proposal.

§ 6. Wording

Full formal wording will be provided in a future revision after LEWG review of the design direction. The proposed additions include:

§ 6.1. [utility.rebind] Generalised element-type conversion

§ 6.1.1. Type trait `rebind_t`

A type trait that computes the result of rebinding a type `T` to use element type `U`:

```
template<typename U, typename T>
using rebind_t = decltype(rebind<U>(std::declval<T>()));
```

Remarks: `rebind_t<U, T>` is well-formed only if `rebind<U>(std::declval<T>())` is well-formed.

§ 6.1.2. Customisation point object `rebind`

A customisation point object `std::rebind` that converts the element type of containers and other types with uniform element types.

Customisation point behaviour:

- Performs ADL lookup for `rebind` in the associated namespaces of the argument type
- Falls back to standard library overloads if no ADL candidate is found
- Returns a new object of the rebound type with elements converted from the source

Standard library overloads for:

- `std::array<T, N> → std::array<U, N>`
- `std::vector<T, Allocator> → std::vector<U, rebind_alloc_t<Allocator, U>>`
- `std::deque<T, Allocator> → std::deque<U, rebind_alloc_t<Allocator, U>>`
- `std::list<T, Allocator> → std::list<U, rebind_alloc_t<Allocator, U>>`
- `std::forward_list<T, Allocator> → std::forward_list<U, rebind_alloc_t<Allocator, U>>`
- `std::complex<T> → std::complex<U>`

Constraints:

- The conversion from `T` to `U` must be valid (using standard C++ conversion rules)
- For allocator-aware containers, `rebind_alloc_t<Allocator, U>` must be valid

Effects:

- Creates a new object with elements converted element-wise from the source
- Element conversion uses standard C++ conversion semantics (equivalent to `static_cast<U>(t)` for each element)
- Preserves container size for sequence containers
- For allocator-aware containers, uses the rebound allocator

Complexity:

- Linear in the number of elements

Exception guarantees:

- Basic exception guarantee: if an element conversion throws, the function exits via exception and the source object remains valid

§ References

§ Informative References

[P3045R1]

Mateusz Pusz, Dominik Berner, Johel Ernesto Guerrero Peña, Charles Hogg, Nicolas Holthaus, Roth Michaels, Vincent Reverdy. *Quantities and units library*. 22 May 2024. URL: <https://wg21.link/p3045r1>